CSI

Configuración personal:

nombre equipo	ediez@maude	160202	
correo thunderbird	ediez@c17.net	160202	
ssh		172223	
cuenta gitHub	ediez@c17.net	160202Es@	
cuenta gitHub mia	estdiesai83@gmail.com	160202Es@	
bitbucket (Atlassian)	ediesai (usuario)	160202Es@	bucked name : ediez@maude
fichme	ediez@c17.net	160202Es@	web para fichar

ALARMA

clave: **1602** clave coacción: 1604

botón derecha **poner** botón izquierda **quitar**

botón mitad negro/blanco -> alarma parcial, la alarma solo esta puesta para los accesos (puertas, ventanas) pero te puedes mover.

docker Hub conectado automáticamente con la cuenta de GitHub.

Hay parte del código en bicbucket. **Bitbucket** es una plataforma de alojamiento basada en la web diseñada para proyectos que utilizan el sistema de control de versiones **Git**. Es propiedad de **Atlassian**.

Si Bitbucket es el lugar donde guardas el código fuente, **Docker Hub** es el lugar donde guardas las "aplicaciones terminadas" listas para ejecutarse.

Es el registro oficial en la nube de **Docker**, una biblioteca gigante que contiene imágenes de contenedores. Imaginalo como una **App Store** o un **Google Play**, pero en lugar de apps para el móvil, descargas entornos de software completos.

Yakuake: Consola que utilizan: F12 o desde la terminal yakuake&

Yakuake (Yet another Kuake) es un emulador de terminal desplegable diseñado para el entorno de escritorio KDE en Linux, incluido Debian. Su principal característica es que permanece oculto y aparece deslizándose desde la parte superior de la pantalla al presionar una tecla de acceso rápido (por defecto **F12**), lo que permite un acceso rápido a la línea de comandos sin cambiar de ventana.

Estructura proyecto symfony

c17 (workspace) -> www.c17.net

ReadModel (Ingles)

WriteModel (español)

Hay 3 directorios con el archivo composer.lock.

1. **ansible:** proceso de despliege
    1. deploy_prod: la de producción
    2. deploy_staging: la de preproducción
2. **web/**
    1. **portal**: aplicación para los usuarios
        1. **teenants:** (cliente), contiene personalizaciones por cliente. Ej: logo, particularidades específicas. Cada cliente tiene una carpeta. Aquí los clientes son siempre instituciones, que pueden tener 1 o varias bibliotecas.

           Cada teenant tiene sus usuarios, si un usuario cambia de teenant, se tiene que volver a registrar (NO lo movemos a otro).
    2. **sf17**: orientada a las bibliotecas, contiene la llamada al código antiguo (r17) desde el controlador SegullAppWrapper.
    3. **r17**: proyecto antiguo (LEGACY)
3. **lib/**
    1. **c17-cqrs-bundle:** para que funcione (autocableado) de c17-cqrs
    2. **c17-cqrs:** conceptos básicos. Ej: command bus, command queue
    3. **c17-db-test**: está definida la base de datos de desarrollo para hacer las pruebas.

       Con el comando: composer pre-test y composes pre-dev restablece las bases de datos. Ejecuta el bin para limpiar las bds. Los datos de desarrollo y test son los mismos, pero son dos bases de datos.
        1. /files/fixtures/003-usuarios.yaml contiene los datos de los usuarios.
    4. **c17-domain-bundle**: para que funcione (autocableado) de c17-domain
    5. **c17-domain-doctrine**
    6. **c17-domain**: Es LEGACY por ahora no lo vemos, se tocará si hay algún error, esto ha ido avanzando a c17_model.
    7. **c17-model-bundle**
    8. **c17-model/src:** Es el nucleo, aquí va lo nuevo que vamos haciendo
        1. **Application**: casi siempre hay un archivo paralelo en cada carpeta de Command y CommandHandler
            1. **Command**
            2. **CommandHandler**
        2. **Dto**: (Data Transfer Object), parecido a ValueObject pero NO tienen Identidad, pasan información de un lado a otro.

3. **Event**: Ej: cuando se da de alta y se quiere programar la baja, entonces se crea el evento
    1. **DomainEvent:** (se notifican dentro del sistema, todavía no ha terminado (modelo))
    2. **IntegrationEvent:** (se notifican hacia fuera, ej: enviar email, este es cuando todo ha terminado y no hay error)

    Del domain pasa al integration.
4. **EventHandler**: Ej: ProgramadorBaja.php
    1. **DomainEventHandler**
    2. **IntegrationEventHandler**
5. **Infraestructure**:
    1. **Doctrine:**
    2. **Elastica:**
        1. **Projection/IllRequest**
            1. FolderAgencyRowStore.php (biblioteca)
            2. FolderRequesterRowStore.php
        2. **ReadModel/IllRequest**
            1. AgencyBalanceReader.php
            2. FolderRowReader.php
6. **Projection**: (son proyecciones de los datos), nosotros sólo tenemos 1 proyección, cuando hacemos una petición, cada biblioteca ve una proyección diferente (Máx 3 proyecciones)
7. **ReadModel (Ingles)**
8. **ValueObject**: objeto sin identidad, se identifica por sus propiedades (Reglas de Validación), siempre tiene que tener todo.
    1. **BillingData**
    2. **CoverageYear**: (cobertura)
    3. **EmailAddress:**
    4. **Isbn:**
    5. **Issn:**
    6. **Locale:**
    7. **Url:**
9. **WriteModel (español)**

9. **c17-openurl-bundle**
10. **c17-shared-bundle**: son bundles que se utilizan en el desarrollo nuevo para que se pueda utilizar tanto en portal como en sf17.
11. **papi:** autentificación (enlace simbólico, ya lo veremos)

4. **patches:** es las versiones LEGACY se han añadido parches para que código antiguo funcione con las versiones nuevas de php.

Información importante

- **archivos .yaml**: es un estándar de serialización de datos diseñado para ser legible por humanos. Se utiliza para archivos de configuración en el desarrollo de software moderno. A diferencia de XML o JSON no utiliza llaves `{}` ni etiquetas `<>` su "magia" reside en la **indentación** (espacios en blanco).

  Legibilidad: Parece una lista de notas organizada.

  Basado en espacios: El nivel de jerarquía se define por cuántos espacios tiene una línea a la izquierda. **Importante:** Se deben usar espacios, no tabuladores.

  Extensiones comunes: `.yaml` o `.yml`.

  2. Estructura y Sintaxis

  YAML utiliza principalmente tres tipos de estructuras:

  Mapas (Clave: Valor)

  Es la forma más básica de guardar información.

```yaml
nombre: "Proyecto Alpha"
version: 1.2
estado: activo
```

  Listas (Colecciones)

  Se definen usando un guion `-` seguido de un espacio.

```yaml
lenguajes:
  - PHP
  - Python
  - JavaScript
```

  Objetos Anidados

  Combinando ambos para crear estructuras complejas.

```yaml
servidor:
  puerto: 8080
  entorno: produccion
  usuarios:
    - admin
    - developer
```

  3. ¿Para qué se usa?

  Si ves un archivo `.yaml`, lo más probable es que estés ante uno de estos escenarios:

  - **Docker:** El archivo `docker-compose.yml` define cómo deben correr tus contenedores.
  - **Kubernetes:** Se usa para describir el estado deseado de los clusters.
  - **GitHub Actions:** Las automatizaciones (pipelines) se escriben en YAML.
  - **Frameworks (Symfony, Spring):** Configuración de servicios y parámetros.

  4. YAML vs JSON

Aunque ambos sirven para lo mismo, tienen personalidades distintas:

Característica	YAML	JSON
Legibilidad	Muy alta (limpio).	Media (muchos símbolos).
Comentarios	Permite usar `#`.	No permite comentarios.
Complejidad	Sensible a espacios (peligroso).	Más rígido pero robusto.
Uso principal	Configuración manual.	APIs y transferencia de datos.

Docker

Es un sistema de contenedores de transporte de software, empaqueta las bases de datos, las librerias....

1. ¿Cómo funciona? (La analogía del Barco)

Imagina un barco de carga. No importa si llevas coches, comida o muebles; todo va dentro de **contenedores estándar**. El barco (tu ordenador o servidor) solo se preocupa de cargar los contenedores, y lo que hay dentro está aislado de lo demás.

- **Imagen:** Es la "receta" o el molde. Contiene el sistema operativo ligero, el código y las dependencias. No cambia (es inmutable).
- **Contenedor:** Es la imagen en ejecución. Es como una "máquina virtual" pero mucho más ligera y rápida porque comparte el núcleo del sistema operativo principal.

2. ¿Por qué es mejor que una Máquina Virtual?

Característica	Máquina Virtual (VM)	Contenedor (Docker)
Peso	Gigabytes (incluye un SO completo)	Megabytes (solo lo necesario)
Velocidad	Tarda minutos en arrancar	Arranca en segundos
Rendimiento	Consume muchos recursos (RAM/CPU)	Muy eficiente, casi nativo

A diferencia de una máquina virtual, Docker no emula un hardware completo, comparte el núcleo (kernel) de tu sistema operativo.

3. Los 3 pilares de Docker

A. El Dockerfile (La receta): define el entorno

Es un archivo de texto donde escribes las instrucciones paso a paso para construir tu imagen. (ej: "Instala PHP 8.2", "Copia estos archivos", "Abre el puerto 80").

Dockerfile

```
FROM php:8.2-fpm
RUN apt-get update && apt-get install -y libpq-dev
COPY . /var/www/html
```

Imagen: (El paquete) es el resultado de "cocinar" el Dockerfile. Es un archivo pesado que contiene todo lo necesario para ejecutar la app. Es inmutable NO CAMBIA.

Contenedor: (La instancia): Es la imagen "cobrando vida". Es el proceso que realmente se está ejecutando en tu ordenador. Se puede levantar 10 contenedores iguales a partir de una sola imagen.

B. Docker Hub

Es el "GitHub" de las imágenes. Si necesitas una base de datos MySQL, no la instalas en tu PC; simplemente haces un `docker pull mysql` y ya tienes una imagen oficial lista para usar.

C. Docker Compose

Como vimos antes, es la herramienta para gestionar varios contenedores a la vez (por ejemplo: uno para PHP, otro para MySQL y otro para Nginx) usando un solo archivo `.yml`.

4. Ventajas clave para ti

1. **Aislamiento:** Puedes tener un proyecto en PHP 7.4 y otro en PHP 8.3 en el mismo ordenador sin que choquen entre ellos.
2. **Portabilidad:** Si un compañero se une al equipo, solo hace `docker compose up` y tiene exactamente el mismo entorno que tú en 2 minutos.
3. **Limpieza:** Tu sistema operativo real no se llena de "basura" de instalaciones de bases de datos o servidores; todo vive (y muere) dentro de los contenedores.

Arrancar docker: `docker compose up -d`, donde:

- **docker compose**: llama a la herramienta de orquestación de Docker que lee el archivo docker-compose.yml
- **up**: le dice a docker que realize tres pasos:
  - **descargar** las imágenes (si nos las tiene)
  - **crear** los contenedores.
  - **Arrancarlos**
- **-d** (detached mode): esta es la clave. Ejecuta los contenedores en segundo plano, si no se pone se ven todos los logs en tiempo real y, si cierras la terminal o pulsas `Ctrl+C`, los contenedores se detendrán.

¿Qué sucede cuando lo ejecutas?

Cuando lanzas este comando, Docker sigue este orden lógico:

1. **Lectura del YAML**: Busca el archivo `docker-compose.yml` en el directorio actual.

2. **Creación de Red (Network)**: Crea una red virtual interna para que todos los servicios del archivo puedan hablar entre sí usando sus nombres (ej: que PHP se conecte a `db` en lugar de a una IP).
3. **Pull de imágenes**: Si no tienes las imágenes descargadas localmente, las baja del Docker Hub.
4. **Creación de Volúmenes**: Crea los espacios de almacenamiento persistente si los definiste.
5. **Arranque de Contenedores**: Lanza los contenedores respetando el orden (si usaste `depends_on`).

3. ¿Cómo se configura? (Paso a paso)

Para configurar un entorno de desarrollo (por ejemplo, para PHP/Symfony), normalmente seguimos estos pasos:

Paso A: El Dockerfile (Define tu entorno)

Crea un archivo llamado `Dockerfile` en la raíz de tu proyecto:

Dockerfile

```
# 1. Usamos una imagen oficial de PHP como base
FROM php:8.2-fpm

# 2. Instalamos extensiones necesarias (ej: para bases de datos)
RUN docker-php-ext-install pdo pdo_mysql

# 3. Definimos dónde estará nuestro código dentro del contenedor
WORKDIR /var/www/html
```

Paso B: Docker Compose (Orquesta tus servicios)

Si tu aplicación necesita una base de datos, no la metes en el mismo contenedor que PHP. Creas un archivo `docker-compose.yml` para conectarlos:

```yaml
services:
  # Servicio de PHP
  app:
    build: .
    volumes:
      - .:/var/www/html
    networks:
      - mi-red

  # Servicio de Base de Datos
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
    networks:
      - mi-red

networks:
```

```
  mi-red:
```

Paso C: Encendido

Una vez configurados los archivos, solo necesitas la terminal:

1. **Construir:** `docker compose build` (lee el Dockerfile y crea la imagen).
2. **Levantar:** `docker compose up -d` (arranca PHP y MySQL).

---

4. Ventajas de esta configuración

- **Aislamiento total:** Si borras el contenedor, tu ordenador queda limpio. Nada se instala "realmente" en tu sistema operativo principal.
- **Volúmenes:** En el archivo `.yml`, la línea `- .:/var/www/html` hace magia: permite que los cambios que hagas en tu código en VS Code se vean **instantáneamente** dentro del contenedor.
- **Redes:** El contenedor `app` puede hablar con el contenedor `db` simplemente usando el nombre `db` como si fuera una dirección web.

Comandos útiles relacionados

Si acabas de lanzar un `up -d`, estos comandos te servirán para gestionar lo que está pasando:

- **`docker compose ps`**: Para ver si los contenedores están realmente arriba (`Up`) o si han fallado.
- **`docker compose logs -f`**: Para ver los logs aunque estén en segundo plano (muy útil si algo no carga).
- **`docker compose stop`**: Detiene los contenedores pero los mantiene creados.
- **`docker compose down`**: **Cuidado con este:** Detiene los contenedores y los **elimina**, junto con la red (pero no borra los volúmenes de datos).

Utilizar proyecto

Para entrar en la raiz del proyecto: `docker compose up -d` (-d = detach) , se coloca automáticamente en la raiz del proyecto.

```
user@
```

Para poner la terminal: se utiliza para poner un comando dentro de un contenedor que ya está corriendo, pero forzando una identidad de usuario específica.

```
docker compose exec -u 1000:1000
```

Desglose del comando:

- **`docker compose exec`**: Indica que quieres entrar a un contenedor activo. A diferencia de `run`, este comando no crea un contenedor nuevo, sino que se "mete" en uno que ya existe.

- `-u` (o `--user`): Es la bandera para especificar el usuario.
- `1000:1000`: Define el **UID** (User ID) y el **GID** (Group ID).
  - El primer `1000` es el ID del usuario.
  - El segundo `1000` es el ID del grupo.

¿Por qué es importante el 1000:1000?

En la gran mayoría de las distribuciones de Linux (incluyendo Ubuntu, Debian y las imágenes comunes de Docker), el **ID 1000** es el que se le asigna por defecto al **primer usuario no-root** que se crea en el sistema.

¿Para qué se utiliza principalmente?

1. **Evitar problemas de permisos (Permissions Mismatch):** Si tu usuario en tu computadora física (host) tiene el ID 1000 y creas archivos dentro del contenedor como `root` (que es el default de Docker), luego no podrás editar esos archivos desde tu editor de código fuera del contenedor porque "pertenecen a otro dueño". Al usar `-u 1000:1000`, los archivos que generes dentro pertenecerán a tu usuario de fuera.
2. **Seguridad:** Ejecutar procesos como `root` dentro de un contenedor es una mala práctica de seguridad. Si el proceso se ve comprometido, el atacante tiene privilegios totales. Forzar un usuario con ID 1000 limita lo que se puede hacer.

La estructura de desarrollo queremos que sea parecida a la de producción por ello utilizamos enlaces simbólicos.

Siempre desde docker:

levantar docker: `docker compose up -d`(-d detach)

para entrar en la máquina: `compose exec -u 1000:1000 php bash`

parar docker: `compose down`

Para ejecutar los 3 comandos especificos, colocarnos en la raiz de la aplicación Ej: `cd web/portal` :

```
composer fix-devps
composer pre-dev
composer pre-test
```

Cuando se crea una etiqueta:

1. Llama al controlador (utilizan objetos): c17_shared_bundle/src/Controller/Ill17/IllRequest/IllRequestActionController.php -> al método **tag**.

   ```
   #[IsGranted(IllRequestActionVoterAttribute::TAG->value, 'illRequest')]
       #[Route(path: '/{id<%routing.int%>}/tag', name: '_tag', options:
   ['expose' => true])]
   ```

```
    public function tag(Request $request, IllRequest $illRequest,
LibraryInterface $library): Response
    {
        $commandBuilder = new TagIllRequestCommandBuilder($illRequest,
$library);
        $form = $this->createForm(TagIllRequestFormType::class,
$commandBuilder);
        $form->handleRequest($request);

        if ($form->isSubmitted() && $form->isValid()) {
            $this->application->execute($commandBuilder->buildCommand());

            $this->addFlash('success', 'message.ill_request_tagged_ok');

            return $this->turboAwareRedirectToRoute($request, 'app_c17_ill-
request', ['id' => $illRequest->getId()]);
        }

        return $this->render(
            '@C17Shared/ill17/ill-requests/actions/_tag.html.twig',
            [
                'form' => $form,
                'illRequest' => $illRequest,
            ]
        );
    }
```

LibraryInterface $library retorna un ReadableCollection

1. Crea el commandBuilder con el TagIllRequestCommandBuilder, obtiene el id de la biblioteca la petición (biblio Origen)
2. Crea el formulario (create form)
3. Asigna el handle (sería la etiqueta en este caso)
4. ejecuta el command builder
5. muestra mensaje

2. Crea un comando y lo manda a ejecutar.
3. Busca el handler que en este caso es Etiqueta y le paso el comando.
4. Hace el sql y manda un evento a la grabadora de Eventos DomainEvent. Todo esto se hace en la misma transacción, para asegurarnos de que o se hace todo o nada.
5. Hasta que no termina la transacción no se hacen los eventos de Integración.
6. En los modelos de lectura relacionamos con objetos y en los modelos de escritura , en lugar de utilizar objetos como tal utilizamos identificadores, porque queremos que ese proceso sea más atómico, con los parámetros imprescindibles.

Eventos:

- **Eventos de dominio:** están en la misma transacción.
- **Eventos de integración:** no se ejecutan hasta que no termina la transacción.

Cuando desde la web pulsamos en Etiqueta envía una petición, cada biblioteca tiene sus etiquetas.

En nuestro código en c-17-model/src/WriteModel/ cada directorio suele tener un agregado, algún caso especial tiene dentro de un directorio 2 agregados. EJ: cuentaUsuario sería un agregado, en cambio en FondoRevista hay dos agregados (fondo y revista).

Cuando tienen una agregado, tienen un archivo nombreAgregado y otro nombreAgregadoId. Ej: cuentaUsuario

El modelo de lectura de doctrine está por atributos

```
#[ORM\Table(name: 'libros')]
#[ORM\Entity(repositoryClass: BookRepository::class, readOnly: true)]
class Book implements BookInterface
{
    #[ORM\Id]
    #[ORM\Column(type: 'integer')]
    #[ORM\GeneratedValue(strategy: 'NONE')]
    private int $id;

    #[ORM\Column(type: 'string', length: 512, name: 'titulo')]
    private string $title;
    ....
```

El modelo de escritura de doctrine está por archivo .xml (c17-model/src/infraestructure(Doctrine/ORM/WritteModel/Mapping)) Ej: biblioteca.Biblioteca.orm.xml

```
<doctrine-mapping
    xmlns="http://doctrine-project.org/schemas/orm/doctrine-mapping"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://doctrine-project.org/schemas/orm/doctrine-
mapping https://www.doctrine-project.org/schemas/orm/doctrine-mapping.xsd"
>

    <entity name="C17\Model\WriteModel\Biblioteca\Biblioteca"
table="biblioteca" repository-
class="C17\Model\Infrastructure\Doctrine\Orm\WriteModel\BibliotecaRepositorio">

        <id name="id" column="organisation_id" type="integer">
            <generator strategy="NONE" />
        </id>

        <many-to-one field="institucion" target-
entity="C17\Model\WriteModel\Institucion\Institucion">
            <join-column name="institucion_id" referenced-column-name="id" />
        </many-to-one>

    </entity>

</doctrine-mapping>
```

Aquí se dice los campos que quiere.

Las **Entidades de Doctrine**, no tiene que ver con lo que nosotros queremos mostrar. Ej: en la tabla preferencias solo utilizamos en el código locale y email, no todos los campos que hay en la bd.

FORMULARIOS

Usamos un CommandBuilder para poder cambiar un formulario, porque construye un comando (puede construir todos los que necesite, pero ese comando es immutable).

El formulario no ataca directamente al comando, sino al commandBuilder. Se cambia al objeto al que engloba el campo del formulario.

** Para que todo el funcionamiento esté OK Siempre que se hace un POST hay que hacer un `$this ->redirect, no con un render, para que no se pueda dar atrás.

Arquitectura Hexagonal: por un lado en la lectura tiene la interfaz y la implementación está en doctrine.

Llamamos reader a los que llama a las consultas.

En c17-model/src/infraestructure se hacen las implementaciones con ORM, Dbal o Elastic

Comandos específicos

Se ejecutan siempre en la raiz del proyecto.

Creados por nosotros, están guardados en el proyecto en el composer.json principal del proyecto en "scripts"

- **composer fix-deps:** arregla las dependencias.
- **composer pre-dev:** prepara el entorno de desarrollo.
- **composer pre-test:** prepara el entorno de test, (limpia las bases de datos)
- **composer fix-cs:** organiza el código.
- **composer ci:** ejecuta todas las pruebas (se tiene que ejecutar siempre antes de subir una rama, es decir se hace git **add, composer ci y luego el git commit)**.

Composer gestiona las dependencias de php mediante dos archivos:

- **composer.json (El mapa)**: es el archivo que tú creas y editas. Define los requisitos generales de tu proyecto.
    - Propósito: indica que librerías quieres y qué versiones estás dispuesto a aceptar.

    - Versiones: suele usar rangos flexibles. Por ejemplo, `"guzzlehttp/guzzle": "^7.0"` significa que aceptas cualquier versión desde la 7.0 hasta antes de la 8.0.
    - Contenido: además de las dependencias (**require**), contiene metadatos como el nombre del proyecto, la licencia, scripts de automatización y la configuración de Autoload.
- **composer.lock (El contenido):** Si existe un `composer.lock`, el comando `composer install` ignorará rangos del `json` y descargará las versiones fijas que dicta el `lock`.

Comparativa rápida

Característica	composer.json	composer.lock
Quién lo crea	El desarrollador (tú).	Composer (automático).
Versiones	Flexibles (ej. `^2.1`).	Exactas (ej. `2.1.4`).
Cuándo cambia	Cuando añades o quitas paquetes.	Cuando corres `composer update`.
¿Se sube a Git?	**Sí**, siempre.	**Sí**, siempre (excepto en librerías).

¿Cuándo usar cada comando?

- **`composer install`**: Se usa cuando descargas un proyecto por primera vez o cambias de rama. Busca el `composer.lock` para replicar el entorno exacto. Es el que debes usar en **Producción**.
- **`composer update`**: Se usa cuando quieres subir de versión tus librerías. Ignora el `lock`, busca versiones más nuevas que encajen con tu `json` y luego **actualiza** el `lock` con los nuevos datos.

> **Regla de oro:** En un entorno profesional, siempre sube ambos archivos al repositorio. De lo contrario, podrías encontrarte con el clásico "en mi computadora funciona, pero en el servidor no" debido a una pequeña diferencia de versiones.

Configuración sistema

En VS Code: instalar todas las extensiones predefinidas y container tools (diseñada para que no tengas que salir del editor para gestionar tus contenedores)

Utilizamos:

elasticsearch: motor de búsqueda "bd".

elasticvue: utilizamos la versión 8. Cada columna es una tabla. Las bandejas son las que utilizan elacticvue, una vez que se hace clic, ya si se va a la BD.

Las bandejas son el objeto grande con las filas con las pestañas (pendiente,...) en cualquiera de las dos apps.

mailer (mailpit): para el correo. el correo no sale del sistema, puerto 8025

adminer: entorno gráfico para ver las bases de datos, puerto 8080.

GIT

- **git stash:** guarda los cambios de manera temporal sin tener que hacer un commit para limpiar tu espacio de trabajo.

  Imagina que estás trabajando en una funcionalidad nueva y, de repente, surge un error urgente que debes corregir en otra rama.

  - No quieres hacer commit de tu trabajo a medias (porque está incompleto o "roto").
  - Git no te deja cambiar de rama porque tienes archivos modificados.

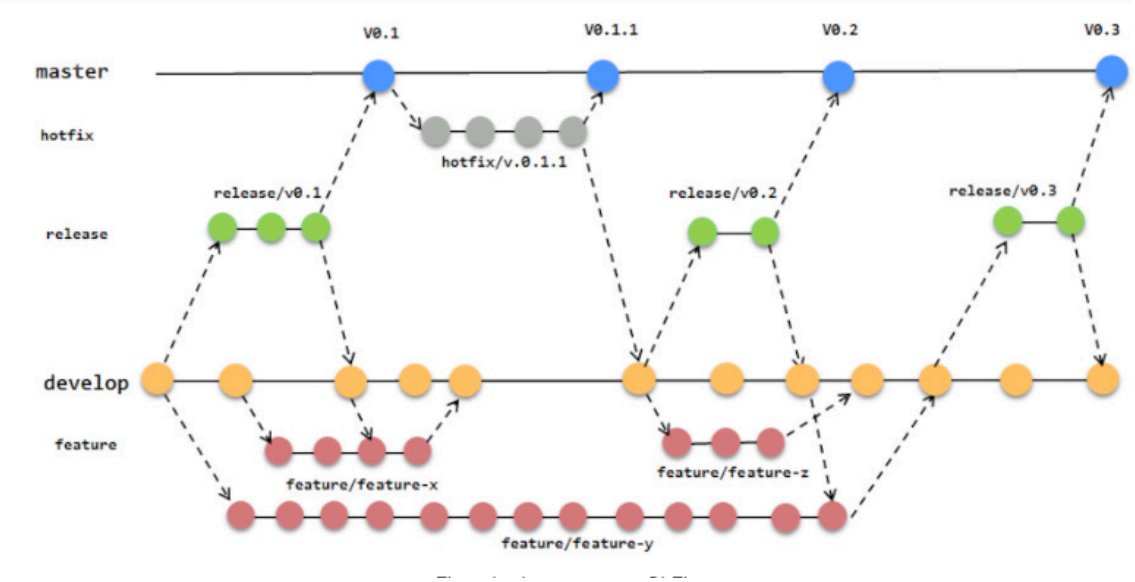
  Solución: Haces un `git stash`, tus cambios se guardan a buen recaudo, tu carpeta se queda limpia, arreglas el error y luego recuperas lo que tenías.
- k --all &: es un visor gráfico de historial para git.
- git flow: comando que permite hacer el funcionamiento de manera sencilla,

  2 ramas principales:

  - main: contiene la última versión de nuestro proyecto y usaremos para desplegar en producción.
  - develop: rama principal de desarrollo

  varias ramas de soporte:

  - feature: para desarrollar nuevas funcionalidades, se crean a partir de la rama develop y al terminar se fusiona con develop.
  - release: se usa para lanzar una nueva versión de nuestro proyecto y se fusionaran tanto con master como con develop.
  - hotfix: se usa para cambios rápidos sobre la rama main y se fusiona con main y develop. Ej: porque haya un error que corregir



develop: rama principal de desarrollo

```
git flow init
git flow feature start prueba
cuando termino git flow feature finish prueba
git flow release start 20260407123400 (nombre fecha inversa)
git flow release finish -n (no usamos tag)
gir merge tool
```

```
git flow hotfix start 20250407135353
git stash pop (recupera los últimos cambios)
git flow hotfix finish -n 20250407135353
```

Para actualizar dos ramas en el servidor (`origin`) al mismo tiempo, pero aplicando **cinturones de seguridad** para no borrar el trabajo de tus compañeros por accidente.

```
git push origin develop main --force-with-lease --force-if-includes
```

Si me quiero bajar los cambios **NUNCA HACER GIT PULL**, para que no haya conflictos en su lugar: git fetch o git rebase.

- **git fetch:** descarga el historial del servidor, pero no toca tu código de trabajo.
- **git rebase**: mueve o combina una serie de commits sobre una nueva base.
- **git merge tool:** utilidad externa que se configura para resolver conflictos de fusión de manera visual. Utilizamos **Kdiff3**: tiene 4 paneles para ver los conflictos.
- **git nah**: resetea todo. No es un comando oficial de Git, sino un alias (atajo personalizado) popular entre desarrolladores. Se utiliza para decir: "Olvida todo lo que acabo de hacer, limpia el desorden y vuelve al estado del último commit". (Es el botón del pánico para cuando has hecho cambios experimentales que no funcionan y quieres borrarlos todos de un plumazo)

*Desde VS Code en SOURCE CONTROL nos deja hacer los git add, git commit (es mejor que desde consola por que se ven los cambios que se van a subir). **SIEMPRE HACERLO EN ESTE ORDEN**

```
git add
composer fix-cs
git commit
```

FRONTEND

- **webpack encore:** biblioteca de PHP para symfony para gestionar los assets (recursos) de tu aplicación web: archivos JavaScript, estilos CSS, imágenes y fuentes.
- bootstrap:
  - v3 (la utiliza la versión LEGACY)
  - v5 (lo nuevo app portal)

  Si la barra de abajo está siempre visible es la v5 y si no es la v3.
- **SCSS:** (Sassy Cascading Style Sheets) Es mejor para el anidamiento, luego con nodejs pasa a CSS.

  Es un preprocesador de CSS. Es una versión más potente del CSS convencional. Permite usar herramientas lógicas (variables, funciones, ciclos) para escribir estilos de forma más rápida, organizada y fácil de mantener.
- **Workspaces**: contiene la sf17_legacy

Cuando haya cambios en un fichero, pasos a seguir:

```
git add
composer fix-cs
git commit
```

TEST

- rector: actualiza el código con los cambios de versión
- phpUnit: es lo último que se ejecuta. tenemos dos tipos:
    - portal: ejecuta un navegador real para probar javascript, para eso se utiliza el ejecutable que está en drivers y se ejecuta con composer ci.
- phpXtam: detecta errores para arraglarlos antes de subirlos a producción

Los test se ejecutan en orden aleatorio, tenemos 8 test configurados.

bin/phpunit --exclude

DEBUG

Ponemos interrupciones en el código hay que tener abierto VS Code y el navegador pulsada la mariquita , para ello hay que instalar una extensión de firefox: `Xdebug_ext`

Para lanzarlo en VS Code pulsar F5 y en el navegador pulsar la Mariquita.

debug location -> seleccionar docked

CQS y CQRS

Uno es un principio de buenas prácticas al escribir funciones, mientras que el otro es un patrón para diseñar sistemas completos.

CQS (Command Query Separation) es un principio de diseño de software. Su idea es tan simple como poderosa: **un método deber ser un comando o una consulta pero nunca ambos.**

Para entender CQS, hay que dividir las operaciones en dos categorías estrictas.

- **Commands (Comandos):** acciones que cambian el estado del sistema (modifican la base de datos, cambian una variable, etc), pero no devuelven datos, devuelve `void`.
- **Queries (Consultas):** Son acciones que devuelven datos pero no tienen efectos secundarios (no modifican nada).

En resumen: "Hacer una pregunta no debe cambiar la respuesta". Al leer el código, sabes con certeza que si una función devuelve un valor, puedes llamarla 10 veces y el sistema seguirá igual.

Beneficios de usar CQS:

- **Claridad mental:** Al leer el código, sabes que si un método devuelve algo, es seguro llamarlo. Si no devuelve nada, sabes que algo va a cambiar.
- **Facilidad de testing:** Las consultas son extemandamente fáciles de probar porque no ensucian el estado del sistema

- **Mantenibilidad:** Reduce los "efectos secundarios" sorpresa que suelen causar bugs (error de software) difíciles de rastrear.

CQRS (Command Query Responsibility Segregation)

Es un patrón de arquitectura que lleva el concepto de CQS al nivel de **objetos o servicios**. En lugar de solo separar métodos, separas los modelos de datos para lectura y escritura.

- **Lado de Escritura (Commands):** Optimizado para la validación de reglas de negocio y consistencia.
- **Lado de Lectura (Queries):** Optimizado para la velocidad y la presentación (vistas).

¿Para qué sirve? * **Escalabilidad:** Puedes escalar el servidor de lectura independientemente del de escritura (normalmente se lee mucho más de lo que se escribe).

- **Rendimiento:** Puedes usar una base de datos relacional para escribir y una NoSQL (o una caché) para leer.
- **Flexibilidad:** El modelo que usas para guardar un usuario no tiene por qué ser el mismo modelo complejo que usas para mostrar su perfil con estadísticas.

¿Cómo se utiliza? Generalmente se implementa usando un **Bus de Mensajes** y separando los modelos:

1. **Command:** Un objeto DTO que describe la intención (`CrearPedido`).
2. **Command Handler:** Procesa el comando y guarda en la BD de escritura.
3. **Query:** Un objeto que pide datos (`ObtenerDetallesPedido`).
4. **Query Handler:** Consulta una BD de lectura (que puede ser una versión simplificada de la de escritura) y devuelve un DTO.

CQS vs CQRS (diferencia clave)

Es común confundirlos, pero no son lo mismo:

1. **CQS**: Es a nivel de métodos y clases. Es una disciplina de programación simple.
2. **CQRS** (Command Query Responsibility Segregation): Es un patrón de arquitectura. Lleva el concepto de CQS al extremo, separando incluso los modelos de datos o las bases de datos (uno para lectura y otro para escritura).

Característica	CQS	CQRS
Nivel	Código (Métodos/Funciones)	Arquitectura (Sistemas/Servicios)
Foco	Evitar efectos secundarios	Escalabilidad y rendimiento
Modelos	Comparte el mismo objeto/clase	Modelos de lectura y escritura distintos
Complejidad	Baja (esencial para código limpio)	Alta (solo para sistemas complejos)

¿Cuándo usar cada uno?

- **CQS: Siempre.** Es una regla de oro para que tu código sea predecible y fácil de mantener.
- **CQRS: Solo cuando sea necesario.** Si tu aplicación tiene mucha carga, modelos de lectura muy distintos a los de escritura, o si estás trabajando con microservicios. No lo uses en proyectos pequeños porque añade mucha complejidad innecesaria.

Command y CommandHandler

En el contexto de **Symfony (PHP)**, el patrón Command/CommandHandler es el estándar moderno para manejar la lógica de negocio de forma desacoplada. Symfony facilita esto enormemente a través de un componente llamado **Messenger**.

1. ¿Qué es cada pieza?

El Command (El "Qué")

Es una clase PHP simple (un DTO) que representa una instrucción. Solo contiene los datos necesarios para ejecutar una acción.

- **No tiene lógica.**
- **Es inmutable** (solo tiene propiedades y un constructor).

El Command Handler (El "Cómo")

Es el servicio encargado de ejecutar la lógica cuando recibe un Command específico.

- **Tiene las dependencias:** Aquí inyectas el EntityManager, servicios de email, APIs, etc.
- **Solo hace una cosa:** Ejecuta la acción que dicta el Command.

2. ¿Para qué sirve?

1. **Desacoplamiento:** Tu controlador no sabe *cómo* se registra un usuario, solo envía el "ticket" (Command) al bus.
2. **Reutilización:** Puedes lanzar el mismo Command desde un controlador, desde un comando de consola (CLI) o desde un cron job.
3. **Procesamiento Asíncrono:** Con Symfony Messenger, puedes enviar un Command a una **cola** (RabbitMQ, Redis) para que se procese en segundo plano sin hacer esperar al usuario.

3. ¿Cómo se utiliza en Symfony?

Para usar esto, asegúrate de tener instalado el componente: `composer require symfony/messenger`.

Paso A: Crear el Command

Imagina que queremos registrar un usuario.

```php
// src/Message/RegisterUserCommand.php
namespace App\Message;

class RegisterUserCommand
{
    public function __construct(
        public readonly string $email,
        public readonly string $password,
    ) {}
}
```

Paso B: Crear el Command Handler

Symfony detecta automáticamente los handlers si implementan `AsMessageHandler`.

```php
// src/MessageHandler/RegisterUserHandler.php
namespace App\MessageHandler;

use App\Message\RegisterUserCommand;
use Symfony\Component\Messenger\Attribute\AsMessageHandler;

#[AsMessageHandler]
class RegisterUserHandler
{
    public function __invoke(RegisterUserCommand $command)
    {
        // 1. Aquí va la lógica de negocio
        // 2. Persistir en base de datos
        // 3. Enviar email de bienvenida, etc.
        echo "Registrando a: " . $command->email;
    }
}
```

Paso C: Usarlo en un Controlador

Para "disparar" el proceso, usamos el **MessageBusInterface**.

```php
// src/Controller/RegistrationController.php
namespace App\Controller;

use App\Message\RegisterUserCommand;
use Symfony\Component\Messenger\MessageBusInterface;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class RegistrationController
{
    #[Route('/register', name: 'app_register')]
    public function register(MessageBusInterface $bus): Response
    {
        // Creamos el comando con los datos del request
        $command = new RegisterUserCommand('user@example.com', '123456');

        // Lo enviamos al bus
        $bus->dispatch($command);
```

```
        return new Response('¡Usuario enviado a procesar!');
    }
}
```

4. El Flujo de Trabajo en Symfony

1. **El Bus:** Recibe el mensaje (`dispatch`).
2. **Middleware:** Symfony puede aplicar lógica intermedia (logs, transacciones de base de datos).
3. **Transporte:** Decide si se ejecuta **ahora mismo** (síncrono) o si se manda a una **cola** (asíncrono).
4. **Handler:** Finalmente, el bus encuentra al "manejador" correcto y le pasa el comando.

CQRS se une con Event-Driven Architecture (Arquitectura dirigida por eventos).

Cuando decimos que "los comandos producen eventos", nos referimos a la cadena de causalidad en el sistema: el **Comando** es la intención ("haz esto") y el **Evento** es el resultado ("esto ya pasó").

1. La diferencia conceptual: Intención vs. Hecho

- **Command (Comando):** Es una orden. Puede ser rechazada (si falla la validación o las reglas de negocio).
  - *Ejemplo:* `ConfirmarPedido`.
- **Event (Evento):** Es algo que **ya sucedió** en el pasado. No se puede rechazar, es un hecho histórico.
  - *Ejemplo:* `PedidoConfirmado`.

2. ¿Cómo funciona el flujo con un EventBus?

El **EventBus** es el cartero que lleva la noticia a otras partes del sistema que no sabían lo que el CommandHandler acaba de hacer.

1. **Llega el Comando:** El `ConfirmarPedidoHandler` recibe el comando.
2. **Se ejecuta la lógica:** El handler actualiza la base de datos (pone el pedido como "pagado").
3. **Se dispara el Evento:** Una vez que la base de datos se guarda con éxito, el handler publica un mensaje `PedidoConfirmado` en el **EventBus**.
4. **Reacción:** Otros servicios que están escuchando el EventBus reaccionan de forma independiente:
   - El **Servicio de Email** envía el ticket al cliente.
   - El **Servicio de Logística** empieza a preparar el paquete.
   - El **Servicio de Estadísticas** actualiza el gráfico de ventas.

3. ¿Por qué se hace así? (Desacoplamiento total)

Sin eventos, el `ConfirmarPedidoHandler` tendría que conocer a todos los demás servicios:

```php
// MAL: El handler sabe demasiado
public function handle(ConfirmarPedido $cmd) {
    $this->db->save($pedido);
    $this->emailService->send(...);
    $this->logisticaService->notificar(...);
    $this->statsService->update(...);
}
```

Con eventos y **EventBus**, el handler solo se encarga de lo suyo:

```php
// BIEN: El handler solo cumple la orden y avisa que terminó
public function handle(ConfirmarPedido $cmd) {
    $this->db->save($pedido);

    // El EventBus se encarga de repartir la noticia
    $this->eventBus->publish(new PedidoConfirmado($pedido->getId()));
}
```

4. Resumen técnico en Symfony

Si usas **Symfony Messenger**, esto se traduce en tener dos buses:

1. **CommandBus:** Síncrono. Ejecuta la acción principal.
2. **EventBus:** A menudo asíncrono. Despacha los eventos a "Listeners" o "Subscribers" que realizan tareas secundarias sin bloquear al usuario.

En pocas palabras:

El **Comando** es el detonante y el **Evento** es la onda expansiva que permite que el resto del sistema se entere y reaccione sin que el proceso principal se vuelva lento o complejo.

Lógica de Negocio:

Por el catálogo c17 es como nos conocen.

A partir del catálogo c17 hay varios módulos:

- **C17**+: lanza las peticiones.
- **ILL17:** modulo de préstamos, contiene más info de la petición.
- **B17:** módulo de libros (= c17 pero con libros)
- **K17:** control de revistas en papel. (kardest)
- **P17:** las revistas se suscriben en bloques
- **O17:** los usuarios pueden acceder a la biblioteca cs1.c17

Cuando en la URL www.c17..... poner /index.php/ nos indica que está en la aplicación LEGACY (r17 controlardor SegullApp)

bvcsm: es la de la Comunidad de Madrid

JAMA: es una revista.

PubMed:

Base de datos de Medicina de EEUU con un ID para cada artículo (ese id lo cogen para trabajar).

PMID: Identificados en PubMed (id único y secuencial, es un entero secuencial).

DOI: identificador único para cada artículo, también es una URL única que va al proveedor raiz.

No todo tiene que tener DOI, pero si PMID que es lo que nosotros utilizamos (lo metemos en la web de usuarios para filtrar).

Una vez que se hace una petición en la web c17 aparece en la bandeja de pendientes ordenado por * y luego alfabéticamente.

Agencia: se refiere a una biblioteca.

Instituciones: agrupan bibliotecas. Ej: Consejería de Madrid.

Fondos: cobertura que tiene una biblioteca para una revista. Relaciona una cobertura con una revista.

Colección: hay tres tipos:

- Propia: quieren quitarla, pertenece a una biblioteca y siempre está suscrita.
- Paquetes (colección suscribible): puede pertenecer a una biblioteca o institución y necesita estar suscrita para que aparezca en los fondos.
- Libres: a nosotros como empresa (revistas gratuitas). Se utiliza como añadido a los que están suscritos o son clientes de más tiempo o con más confianza.

Balance: equilibrio respecto a las otras bibliotecas (es la diferencia entre las propias y las ajenas).

Cobertura: es cuando empieza y cuando acaba, (es un año por ejemplo). Ej: años de cobertura de una revista. (1975,1978-89,(1990-93),1998-2002,(2025-23),2025.

*si acaba en . la colección esta cerrada y si acaba en - está abierta

- en papel
- electrónico

fondo-> catálogo colección -> n revistas -> máz 1 papel y 1 electronica.

Cuando se crea se le pone la fecha.

preprint: un artículo puede estar disponible suelto antes de estar en la revista. Las revistas se publican cada cierta peridiocidad, pero

Tupla: conjunto de papel y/o electrónico

Collection Manager: quien es el dueño, es el que lo puede editar (editor).

ISBN: ID de un libro, si se reedita se cambia el ISBN

ISSN: ID de las revistas, hay uno para el papel y otro para el digital. NO cambia el ISSN.

Petición: Una petición puede ser:

- propia:
  - interna
  - externa
- ajena

Dependiendo de quien la vea estará en una bandeja o en otra.

Petición -> Biblioteca A -> lo tengo (interna)

Petición -> Biblioteca A -> NO lo tengo (externa, la pido a otra biblioteca B) *Para la Biblioteca B la petición es ajena.

FeatureFlag: activa o desactiva comportamientos. Ej: algunos usuarios pueden ver la vista nueva y/o antigua.

Backoficce: web donde nosotros vamos implementando acciones para facilitar

Actualización de fondos es con lo que están trabajando ahora.

Pruebas:

Web pruebas:

www.c17.localhost: la web del catálogo para las bibliotecas.

www.csi.c17.localhost: la web de los usuarios (nuestra)

Usuarios pruebas:

Desde la web para navegar y probar

usuario: **MANTON**

usuario: **manager10**: biblioteca de prueba (lo que ven bibliotecario)

Todos los usuarios tienen la misma clave: foobar (coincide con el nombre de la bd)

user100: usuario de prueba de la web de usuarios csi

usuario: **admin** clave: **foobar**

GLOSARIO

Los comando producen eventos (eventbus)

No usamos event sourcing

La estructura de carpetas del event es parecida a la de command (Command y CommadHandler) (Event y EventHandler)

Command bus: hace de intermediario entre quien envia un comando y quien lo procesa.

- Recibe un comando
- Encuentra el handler asociado
- Invoca el handler con el comando.

El **handler** contiene la lógica de procesamiento del comando. Recibe el comando, ejecuta la lógica de negocio y persiste los cambios.

- Crea o recupera el agregado.
- Ejecuta la operación de negocio
- persiste el agregado
- publica los eventos generados.

Algunos comandos pueden producir eventos (EventBus), es importante ya que es el mecanismo que conecta el WriteModel con los ReadModels.

Cuando el agregado genera eventos el EventBus los distribuye a las proyecciones.

- **CQRS**: Comman Query Responsibility Segregatión: separa las operaciones que modifican el estado del sistema (comandos) de aquellas que únicamente consultan información (queries).
- **CQS**: Comman Query Separation: Un método debe hacer una sola cosa: modificar estado o devolver datos, pero no ambos.
- **DDD: Domain-Driven-Desing:** Representa el conocimiento del dominio dentro del código, encapsulando reglas e invariantes del negocio en agregados coherentes.

  Los modelos de lectura tienen que estar sincronizados con los eventos de escritura.

  https://www.netmentor.es/entrada/domain-driven-design

  Contiene el concepto de Domain Events. Un ejemplo de evento de dominio (recorder). Un "dominio" no es el código, es el acto físico de mover paquetes de un punto A a un punto B.

  - **Entidades**: objeto relacionado que **tiene** una **identidad única que persiste en el tiempo** independientemente de si sus atributos cambian.
  - **Agregado**: Tiene ID (se puede identificar). Es el único punto de entrada, si hay que hacer algo es el encargado.
  - **ValueObject**: aspectos del dominio que **no tienen identidad propia**, lo que importa son sus atributos. Si cambias un atributo, tienes un objeto nuevo.
- **Eventos de integración:** se utilizan para comunicar cambios de estado entre diferentes microservicios, sistemas externos

- **DTO** (Data Transfer Object) es un patrón de diseño cuyo único propósito es **transportar datos** entre procesos o partes de un sistema.

  En términos sencillos: imagina que el DTO es un sobre. No le importa la carta que lleva dentro ni quién la escribió; su única función es asegurar que la información llegue de un punto A a un punto B de forma organizada.

  El DTO es un **mensajero**. Su única misión es mover datos de un sitio a otro (por ejemplo, de tu base de datos a una pantalla de celular).

    - **Identidad:** No le importa quién es. Si tienes dos DTOs con los mismos datos, para el sistema siguen siendo solo dos paquetes de datos.
    - **Mutabilidad:** Suelen ser mutables (puedes cambiar sus valores después de crearlos), aunque es buena práctica hacerlos inmutables.
    - **Lógica: Cero lógica**. Solo tiene atributos, *getters* y *setters*.
    - **Uso:** En las fronteras de tu aplicación (API, controladores, servicios externos).

- **Value Object (Objeto de Valor): (VO)** es una pieza de tu **lógica de negocio**. Representa un concepto descriptivo del dominio que no tiene una identidad única, sino que se define por lo que vale.
    - **Identidad por valor:** Si dos Value Objects tienen los mismos atributos, **son el mismo objeto**.
        - *Ejemplo:* Un billete de 10 USD. No te importa el número de serie (identidad), te importa que vale "10" y es "USD". Si lo cambias por otro de 10 USD, tienes lo mismo.
    - **Inmutabilidad:** Esto es sagrado. Un VO **nunca cambia**. Si quieres "modificarlo", creas uno nuevo con el valor diferente.
    - **Lógica:** Tiene **lógica de validación**. Un VO se asegura de que los datos sean correctos desde que nace.
        - *Ejemplo:* Un VO llamado `Email` lanzará un error si intentas crearlo sin un "@".
    - **Uso:** Dentro del núcleo de tu aplicación (el Dominio).

Comparativa Rápida DTO y ValueObject

Característica	DTO	Value Object
Propósito	Transportar datos (Mensajero).	Representar un concepto (Lógica).
Identidad	No tiene identidad propia.	Su identidad es su valor.
Validación	No suele validar nada.	Se auto-valida (siempre es válido).
Cambios	Se pueden cambiar sus campos.	Es 100% inmutable.
Ubicación	Capas externas (API/Red).	Capa interna (Dominio/Negocio).

1. Elasticsearch: El Motor (Backend)

Es un **motor de búsqueda y analítica** distribuido. Imaginalo como una base de datos superpotente diseñada para buscar entre millones de datos en milisegundos.

- **¿Qué hace?** Almacena, busca y analiza grandes volúmenes de datos.
- **¿Cómo se usa?** Principalmente a través de código o peticiones técnicas (API REST). No tiene una "cara" o interfaz visual propia por defecto; es puro código y terminal.
- **Instalación:** Se instala en servidores (locales o en la nube).

2. Elasticvue: La Herramienta Visual (Frontend)

Es una **interfaz de usuario (GUI)** gratuita y de código abierto para gestionar Elasticsearch. No es una base de datos, es una herramienta para que los humanos puedan interactuar con Elasticsearch sin escribir tanto código.

- **¿Qué hace?** Te permite ver tus índices, realizar búsquedas, editar documentos y revisar el estado de tu servidor de Elasticsearch de forma gráfica.
- **¿Cómo se usa?** Se instala como una **extensión de navegador** (Chrome, Firefox) o como una aplicación de escritorio/Docker.
- **Relación:** Necesitas tener Elasticsearch funcionando para que Elasticvue sirva de algo.

Comparativa Directa ElasticSearch y Elasticvue

Característica	Elasticsearch	Elasticvue
Naturaleza	Motor de base de datos / Búsqueda.	Interfaz de usuario (Visualizador).
Función	Almacenar y procesar datos.	Ver y administrar los datos.
Interfaz	Línea de comandos / API JSON.	Botones, tablas y menús visuales.
Analogía	El motor de un coche.	El tablero de instrumentos (velocímetro, luces).

Ansible: proceso de despliege. es una herramienta de código abierto que sirve para automatizar tareas de TI, como configurar servidores, instalar software y desplegar aplicaciones.

```
sudo apt get install ansible
```

Webs para buscar info:

https://siemprelisto.cl/tecnologias/cqrs/07-command-bus/

https://www.netmentor.es/entrada/domain-driven-design